

Building Voice Channels with WebRTC

My experience

Now I want to talk about what I think was my experience of building this. I started with a lot of research about WebRTC technology, which was a big learning curve because it involved backend concepts and networking terms that I was not very familiar with.

I had covered most of them in networking classes at college, but apart from that I was very rusty with that area. I used tons of resources for learning: Reddit posts, LinkedIn posts, Discord developer blogs, and, of course, official documentation.

The first hurdle came up during the planning process. I simply did not know enough to make confident decisions, which made every step challenging. I also wanted to base as much of the project as possible in the Phoenix and Elixir ecosystem.

That turned out to be a very good fit. OTP gave me a way to manage long-running processes and ephemeral state. Phoenix Channels gave me a signaling layer, and ExWebRTC was especially important because it made a server-side WebRTC endpoint possible inside Phoenix and Elixir.

Codex kept insisting on easier options, which challenged my decisions at first. Since I was not very knowledgeable about voice chat, I had to research every recommendation and push back when the easier choice did not match what I wanted to learn.

Another challenge was using Codex through a complicated multi-phase run while keeping the project context intact. The system was being built over more than one hundred commits, so every new conversation needed enough documentation to understand the previous decisions.

Later, after I had made the technology decisions, came the studying phase. The hard part was grasping the idea of a peer-to-peer connection inside a Phoenix project. This might sound silly to someone more experienced, but everything was running on my machine: the browser and the server.

Most learning material explained WebRTC as a browser-to-browser connection. In my case, there was a split client and server side: the browser had one peer connection, and ExWebRTC created the other peer connection inside the Elixir application.

Phoenix Channels were used for browser-to-server signaling. Since LiveView also uses a WebSocket, I initially wondered why I needed another WebSocket connection. It was technically possible to share more of that infrastructure, but keeping the voice signaling in its own Channel made the responsibilities much clearer.

The other important difference was that the WebSocket and the RTC media path are not the same thing. The Phoenix WebSocket is a TCP-based control connection in this deployment, while WebRTC media is normally UDP-based, with TURN providing fallbacks when direct UDP cannot work.

Understanding the networking behind RTP packets was another major challenge. Direct peer-to-peer connections are not always possible because routers use different kinds of NAT and firewalls can block incoming traffic.

I had to learn how ICE gathers and checks possible paths, how STUN helps discover a public mapping, and how TURN relays media when a direct or reflexive path fails. I still do not know ninety-nine percent of the background work, but I now understand the broad architecture.

After all the learning I have done, I have only scratched the surface of WebRTC and ExWebRTC. That is still a much better place to be than where I started.

Architecture

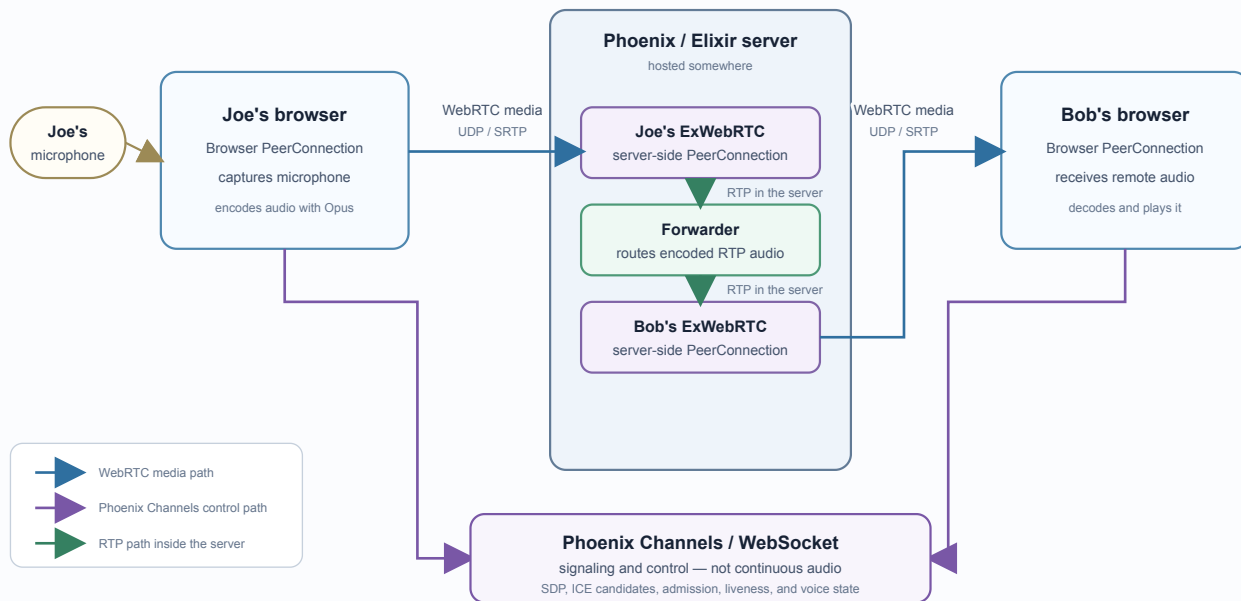
Once I had enough knowledge about how everything was supposed to work, I had to make a few important architectural decisions. The first major decision was to build an audio-only, server-routed system with a maximum of five active Voice Sessions per Voice Channel.

The word “peer” was important here. Each browser has its own browser-side `RTCPeerConnection`, and each browser connects to one server-side `ExWebRTC.PeerConnection`. The browsers do not connect directly to each other.

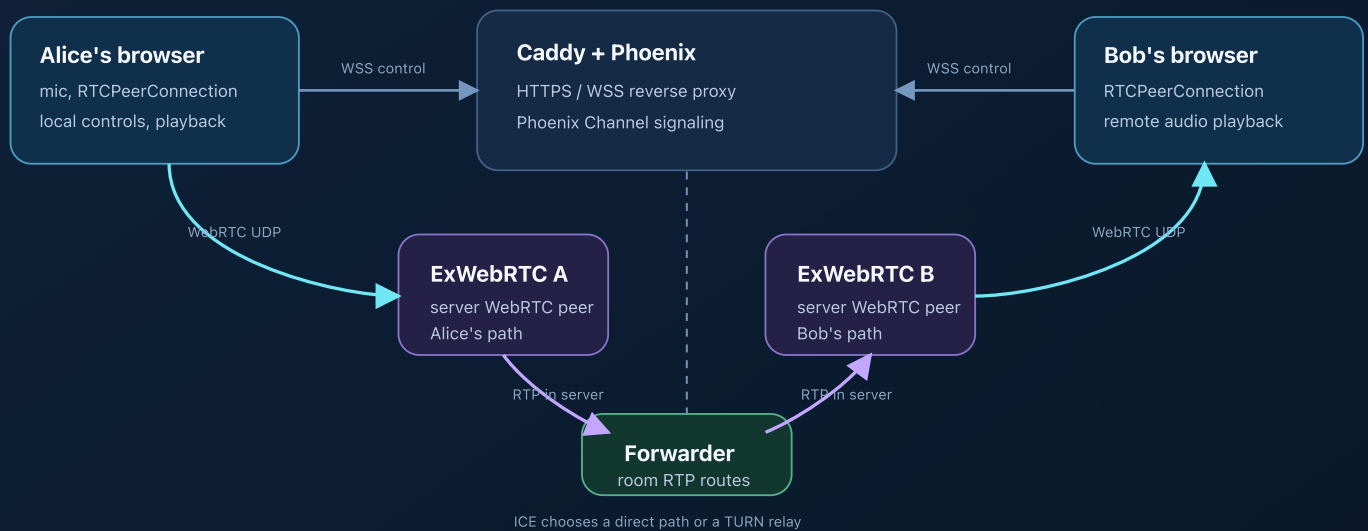
A simplified view of the voice system

The browsers do not connect directly to one another; each browser connects to the server.

No direct browser-to-browser media connection



The simplest way to see the system: browsers connect to the server, the Forwarder routes the media, and Phoenix Channels carry the setup messages separately.



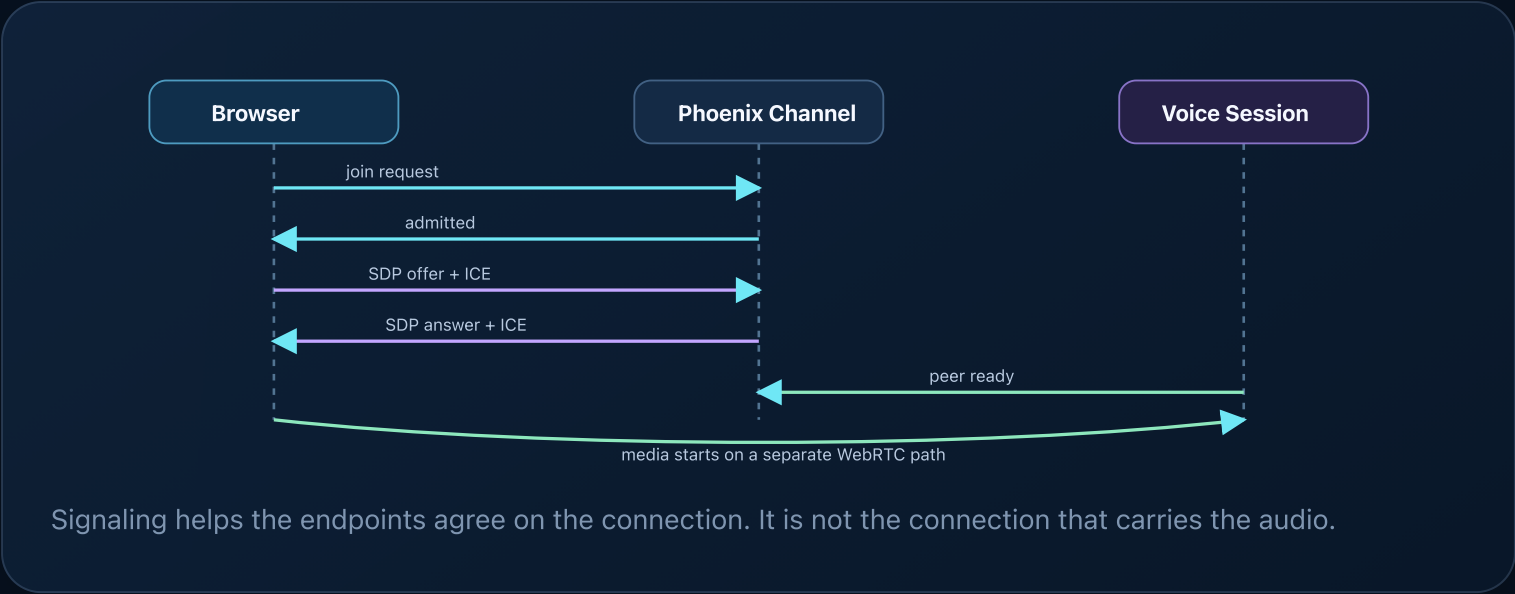
This is the simplified version of the architecture diagrams I made while studying: the control plane and media plane are separate.

Phoenix and the application contexts handle authentication, authorization, durable Voice Channel data, and moderation. OTP handles the live runtime, including room membership, leases, process ownership, and cleanup.

Phoenix Channels are used only for signaling and control. They carry join and leave messages, SDP offers and answers, ICE candidates, liveness renewals, and voice-state updates. They do not carry

continuous microphone audio.

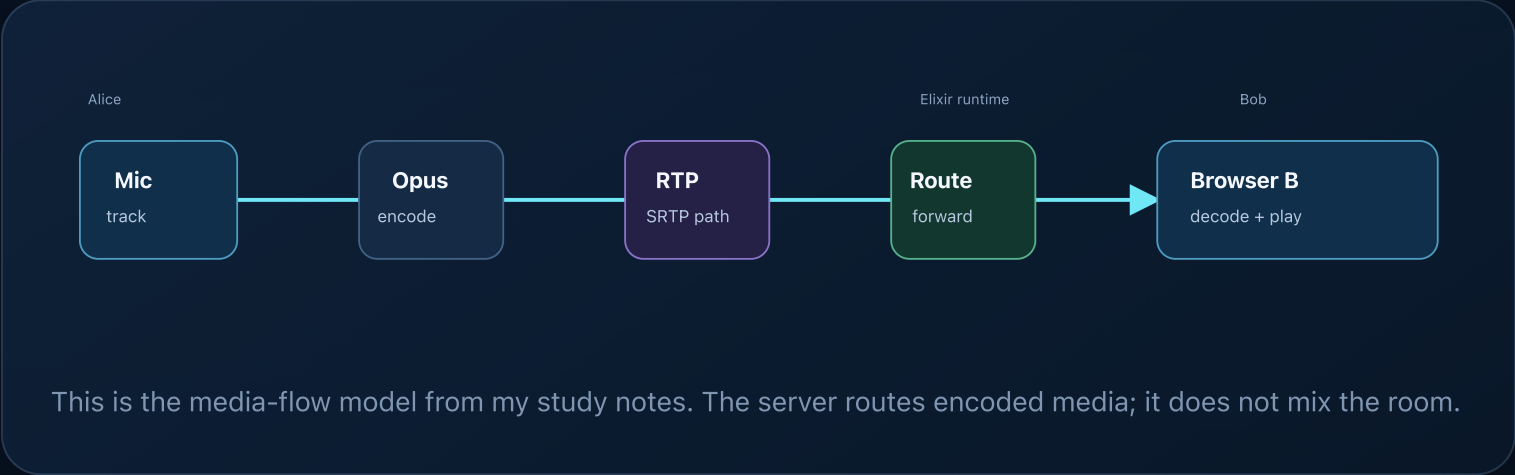
SDP is a text format that describes a multimedia session. It includes media sections, codecs, directions, and transport information. ICE candidates describe possible network paths, and ICE checks which candidate pair can actually work.



The most fascinating thing was that the server side actually has a lot to do with WebRTC, but it does not need to understand the meaning of every audio sample. ExWebRTC terminates the server-side WebRTC connections, while the Forwarder decides where the encoded media should go.

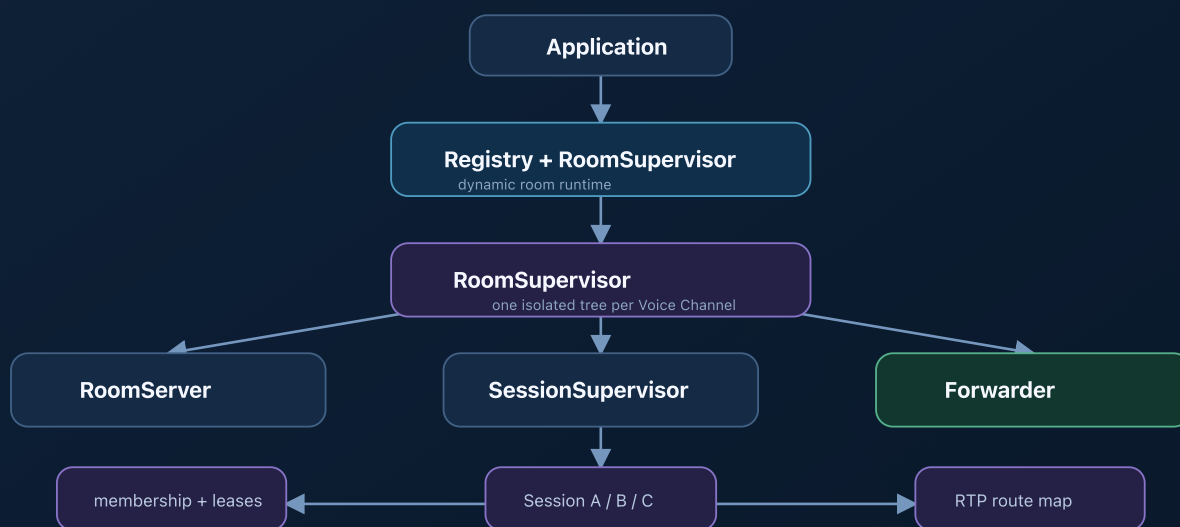
The browser encodes microphone audio with Opus and packetizes it into RTP. WebRTC protects the transport with SRTP. ExWebRTC handles the WebRTC protocol boundary, and the server forwards the encoded RTP packet to another server-side Session.

The receiving ExWebRTC connection sends the media to the other browser. That browser decrypts, depacketizes, decodes, buffers, and plays the audio through the user's speakers. This is why the browser is the part that owns the microphone and playback devices.



Around WebRTC, I built zero direct connections between users. All the traffic forwarding is done through Elixir, thanks to ExWebRTC. That is the part that finally made the “peer-to-peer” explanation make sense to me.

The OTP supervision tree was another important part of the architecture. A durable Voice Channel is stored in Postgres, while a Voice Session is ephemeral runtime state for one admitted browser.

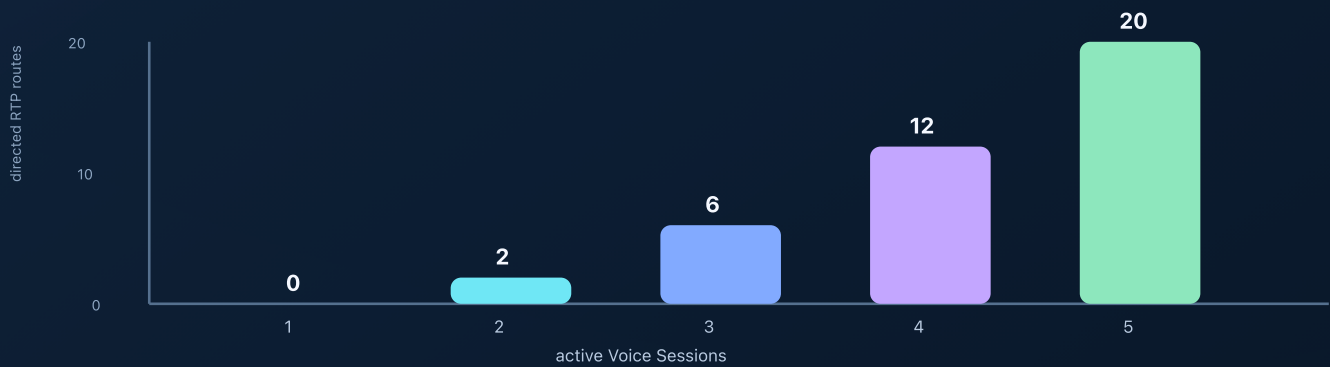


The RoomServer owns membership, the Session owns one WebRTC relationship, and the Forwarder owns media routes.

The RoomServer keeps the canonical membership, capacity, leases, and cleanup state. Each Voice Session owns one server-side peer connection and its negotiation, ICE, readiness, and diagnostics. The Forwarder owns the source and destination RTP routes.

This separation became more important as the project grew. The Phoenix Channel started as the obvious place to put everything, but later phases moved WebRTC ownership into the Session process and kept the Channel as a thinner adapter.

Once I reached the multi-user stage, one number became very satisfying. With five sessions, every user can receive audio from four other users, which creates twenty directed routes. It is five times four, not five times five, because nobody should receive their own audio.



The route count follows $n \times (n - 1)$. The five-user limit made it possible to test the whole route matrix.

The browser preallocates one send-only microphone transceiver and four receive-only audio output slots. This lets the room add or remove participants without renegotiating the whole connection every time.

Development

The first thing I did after the software architecture was clear enough was lay down documentation that would help me use Codex efficiently during development.

This was not something that could be done in a “go make voice, make no mistakes” kind of way or in a couple of loose sessions. The whole process had to be documented so that each new session had enough context to continue without breaking previous decisions.

I made a roadmap with thirteen phases. Under the phases I kept implementation notes, architecture decisions, research documents, a glossary, and context about how the system was supposed to behave.

JULY 27–28

Architecture, durable data, and microphone ownership. I separated durable Voice Channels from runtime Voice Sessions and made the browser responsible for one microphone owner tab.

JULY 31–AUGUST 4

Signaling, the first peer, and RTP echo. The first browser-to-server connection worked locally, followed by a manual headphone proof that RTP could travel through the server endpoint.

AUGUST 4–6

OTP ownership and two-user forwarding. Room, Session, and Forwarder responsibilities became explicit, then one user's encoded audio was routed to another without self-forwarding.

AUGUST 10–11

Five users, controls, and failure behavior. I completed the capped topology, roster and controls, then added leases, recovery deadlines, terminal events, and idempotent cleanup.

AUGUST 26

ICE configuration and private alpha deployment. The local deployment work was certified and the AWS private alpha was brought online, with hosted acceptance still being tested.

Reviewing the code became difficult in the later phases because the system was getting complicated and my lack of experience was catching up with me.

At the same time, the phased approach helped me avoid having every kind of bug at once. I could prove microphone capture, then signaling, then one peer, then echo, then forwarding, and only after that build the room around it.

The local voice path worked sooner than I expected once the pieces were connected. That surprised me because there was also a lot of JavaScript inside the Phoenix project, and that was something I was learning while using it.

The later phases were less about making sound come out of the speakers and more about making the feature behave correctly when users navigate, close tabs, lose network connectivity, reconnect, time out, or log out.

Deployment

Finally, after the development was done, came the deployment phase. This was where I had to make the project work with the complicated networking I had been studying.

I still do not fully grasp everything happening in that layer, but with the help of Codex I managed to get the private alpha running on AWS and test the voice path outside localhost.

The deployment used one ARM64 Ubuntu EC2 instance, a DuckDNS hostname, Caddy, systemd, a private Postgres setup, and Cloudflare Realtime TURN. Phoenix and ExWebRTC ran together in one release on the virtual machine.

Caddy was used as the public HTTPS and WSS reverse proxy. It forwards web and signaling traffic to Phoenix on loopback, but it does not relay the WebRTC UDP media.

The WebRTC media path uses the server's UDP port range directly when ICE selects a host or reflexive route. If a direct route does not work, the media uses a TURN relay. This is separate from the TCP-based WebSocket connection.

PRIVATE ALPHA TOPOLOGY

PIECE	ROLE IN THE DEPLOYMENT
AWS EC2	Runs the Phoenix release, ExWebRTC, and the application runtime on one ARM64 Ubuntu VM.
Caddy	Terminates public HTTP and HTTPS, then proxies web requests and WSS to Phoenix on loopback.
DuckDNS	Provides the stable hostname used by the private alpha.
Cloudflare Realtime TURN	Provides temporary ICE credentials and relay candidates when a relay path is needed.
UDP 50000–50031	Public media range used by the server-side WebRTC endpoints.

Technologies used

The stack looks large when listed together, but every piece had a specific job. Keeping those jobs separate made the system easier to understand.

TECHNOLOGY	WHAT I USED IT FOR
Elixir	The application language and the place where room coordination, session state, and packet routing live.
OTP	GenServers, supervisors, registries, timers, monitors, and restart boundaries for the live voice runtime.

Phoenix	The web framework, authentication boundary, LiveView application, and integration point for the voice system.
Phoenix Channels	The WebSocket signaling and control path for SDP, ICE, admission, liveness, and voice-state events.
ExWebRTC	The server-side WebRTC implementation that terminates browser peer connections and exposes media events.
Browser WebRTC APIs	<code>getUserMedia</code> , <code>RTCPeerConnection</code> , transceivers, ICE, remote tracks, and audio playback.
Opus, RTP, RTCP, SRTP	Audio encoding, packet transport, control and statistics, and protected WebRTC media transport.
STUN and TURN	ICE services that help discover usable network paths and relay media through difficult NAT or firewall boundaries.
Ecto and Postgres	Durable workspace and Voice Channel data, separate from ephemeral live sessions and routes.
Req	The HTTP client used to request temporary TURN credentials from the provider.
Caddy, systemd, AWS, DuckDNS	Reverse proxy, process supervision, virtual machine hosting, and a stable hostname for the private alpha.

Conclusion

Before this project, “WebRTC voice chat” sounded like one feature. It turned out to be a meeting point for browser APIs, networking, security, distributed state, OTP, and product decisions.

The project became manageable when I stopped treating it as one feature and started treating it as a sequence of smaller systems with clear boundaries.

I still have a lot to learn about the networking underneath it, but I am no longer looking at WebRTC as a black box. For a project that started with me not even knowing what WebRTC was, that is probably the most valuable result.